

From Bernoulli Outcomes to Binary Cross-Entropy

Why Logistic Regression Uses Log Loss

Paul Agron

Lecture Notes in Machine Learning and Statistics

Abstract

Binary cross-entropy is often presented as a formula to memorize. This article instead derives it from a sequence of modeling decisions: binary outcomes are described by a Bernoulli distribution; logistic regression assigns a conditional Bernoulli probability to each label; maximum likelihood selects parameters that make the observed labels probable; and logarithms convert independent probability products into additive evidence. We then connect the resulting negative log-likelihood to cross-entropy, entropy, Kullback–Leibler divergence, gradients, convex optimization, and numerically stable computation. Along the way, we distinguish probability from likelihood, a realized log loss from an expected cross-entropy, and the conceptual reason for logarithms from their computational advantages.

The central statement

For Bernoulli logistic regression,

$$\text{binary cross-entropy} = \text{negative Bernoulli log-likelihood}$$

The Bernoulli model does *not* itself contain a logarithm. The logarithm enters when we score or optimize the likelihood. It is the natural functional form when independent probabilities multiply but their information, evidence, or loss should add.

Learning objectives. After reading these notes, a student should be able to:

- define the Bernoulli model, logit, sigmoid, likelihood, log-likelihood, entropy, cross-entropy, and KL divergence;
- derive binary cross-entropy from maximum likelihood;
- explain why a logarithm is conceptually natural, not merely numerically convenient;
- interpret one-observation log loss and dataset-average cross-entropy;
- derive the gradient $p - y$ and explain its optimization behavior; and
- implement the loss stably from logits without evaluating $\log(0)$.

Contents

1	The modeling problem	3
2	The Bernoulli model	3
2.1	One binary observation	3
2.2	Probability versus likelihood	4

3	One observation: log loss and cross-entropy	4
3.1	What does this number measure?	4
3.2	Why the term cross-entropy?	5
4	Why logarithms are the natural choice	5
4.1	Independent probabilities multiply	5
4.2	The base of the logarithm	6
4.3	Logarithms preserve the optimum and improve computation	6
5	Maximum likelihood	7
5.1	Coin-flip example	7
5.2	Likelihood for a logistic-regression dataset	7
6	The shape of binary cross-entropy	8
7	A complete numerical example	9
8	From probability loss to logit loss	9
8.1	The gradient simplifies to $p - y$	10
8.2	Convexity	10
9	Cross-entropy, entropy, and KL divergence (optional)	10
9.1	Why log loss rewards honest probabilities	11
10	Numerical stability	11
10.1	What goes wrong	11
10.2	Stable BCE from logits	12
11	Why not ordinary squared error?	12
12	Assumptions, limitations, and extensions	13
12.1	Conditional independence does not mean independent features	13
12.2	Maximum likelihood depends on the model	13
12.3	Complete separation	13
12.4	Class weighting	13
12.5	Multiclass extension	13
13	Common misconceptions	13
14	Terminology reference	14
15	Summary	15
A	Manifestations of BCE Across Machine-Learning Algorithms	16
A.1	One mathematical template	16
A.2	Direct uses of BCE	17
A.3	Ordinary decision trees: entropy as fitted leaf BCE	18
A.4	Random forests and probability evaluation	18
A.5	Gradient-boosted trees: direct optimization of BCE	19
A.6	Multi-label learning versus multiclass learning	19
A.7	Weighted BCE, focal loss, masks, and soft targets	20
A.8	When BCE is not the appropriate likelihood	21
A.9	Evaluation and interpretation across algorithms	21

1 The modeling problem

We observe a dataset

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n,$$

where $x_i \in \mathbb{R}^d$ is a vector of predictors and $y_i \in \{0, 1\}$ is a binary label. Examples include whether a patient has a disease, whether a customer defaults, or whether an email is spam.

Logistic regression does not directly predict the class label. It first produces a real-valued *linear score*

$$z_i = w^\top x_i + b, \quad (1)$$

and converts it into a probability with the sigmoid function:

$$p_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}}. \quad (2)$$

The model interprets this number as

$$p_i = \mathbb{P}(Y_i = 1 \mid x_i; w, b), \quad 1 - p_i = \mathbb{P}(Y_i = 0 \mid x_i; w, b).$$

The score z_i is also called the *logit*. Indeed,

$$\log \frac{p_i}{1 - p_i} = z_i = w^\top x_i + b. \quad (3)$$

Thus logistic regression assumes that the *log-odds* of class 1 are linear in the predictors. The odds are $p/(1 - p)$; the logit is the natural logarithm of those odds.

Term	Meaning in this article
Score / logit z	Unbounded raw output $w^\top x + b$ before the sigmoid.
Probability p	Model prediction $\mathbb{P}(Y = 1 \mid x; w, b) = \sigma(z)$.
Odds	Ratio $p/(1 - p)$ of class-1 probability to class-0 probability.
Label y	Observed outcome, equal to either 0 or 1.
Parameters θ	Model quantities being fitted; here $\theta = (w, b)$.

2 The Bernoulli model

2.1 One binary observation

A Bernoulli random variable has two possible outcomes. If

$$Y \sim \text{Bernoulli}(p),$$

then

$$\mathbb{P}(Y = 1) = p, \quad \mathbb{P}(Y = 0) = 1 - p.$$

Both cases can be written in one probability mass function:

$$\boxed{\mathbb{P}(Y = y \mid p) = p^y(1 - p)^{1-y}}, \quad y \in \{0, 1\}. \quad (4)$$

The exponents are switches. If $y = 1$, then

$$p^1(1-p)^0 = p;$$

if $y = 0$, then

$$p^0(1-p)^1 = 1-p.$$

In logistic regression, p is not one constant shared by every observation. It depends on the predictors:

$$Y_i \mid x_i \sim \text{Bernoulli}(p_i), \quad p_i = \sigma(w^\top x_i + b).$$

This is a *conditional Bernoulli model*.

2.2 Probability versus likelihood

The same mathematical expression can be viewed in two different ways:

$\mathbb{P}(Y = y \mid p)$	varies the possible data y while p is fixed,
$L(p; y) = p^y(1-p)^{1-y}$	varies the parameter p while observed y is fixed.

The first is a probability mass function. The second is a *likelihood function*. A likelihood is not a probability distribution over the parameter unless additional Bayesian machinery supplies a prior and normalization.

A crucial distinction

Maximum likelihood does not ask, “What is the probability that these parameters are correct?” It asks, “If these parameters were used by the data-generating model, how much probability would that model assign to the data we observed?”

3 One observation: log loss and cross-entropy

Suppose the model predicts $p = \mathbb{P}(Y = 1 \mid x)$ and the observed label is y . The probability assigned to the outcome that actually occurred is

$$r = p^y(1-p)^{1-y} = \begin{cases} p, & y = 1, \\ 1-p, & y = 0. \end{cases} \quad (5)$$

The one-observation *log loss* is

$$\ell(y, p) = -\log r = -[y \log p + (1-y) \log(1-p)]. \quad (6)$$

This is the familiar binary cross-entropy formula.

3.1 What does this number measure?

The quantity $-\log r$ is called the *surprisal* or *self-information* of an event to which the model assigned probability r . A likely event carries little surprise; an event the model considered nearly impossible carries a great deal.

r : probability assigned to what occurred	$-\ln r$	Interpretation
0.99	0.010	almost unsurprising
0.90	0.105	small loss
0.50	0.693	complete binary uncertainty
0.10	2.303	large loss
0.01	4.605	very large loss
0.001	6.908	extreme surprise

The loss does *not* measure ordinary geometric distance such as $|y - p|$. It evaluates the quality of a probabilistic forecast by asking how much probability the forecast assigned to the outcome that occurred.

3.2 Why the term cross-entropy?

Represent the observed label as a point-mass distribution over classes 0 and 1:

$$Q_y = (1 - y, y),$$

and represent the model prediction as

$$P_p = (1 - p, p).$$

For two discrete distributions Q and P , their cross-entropy is

$$H(Q, P) = - \sum_c Q(c) \log P(c). \quad (7)$$

Substitution gives

$$H(Q_y, P_p) = - [(1 - y) \log(1 - p) + y \log p],$$

which is exactly equation (6).

Precise terminology

For one realized label, $-\log P_p(y)$ is most precisely called the *realized logarithmic score* or *one-observation log loss*. It is also the cross-entropy between the model distribution and the empirical point mass placed on that observed label. Cross-entropy in the population sense is an *expectation* over possible outcomes; we develop that distinction in [section 9](#).

4 Why logarithms are the natural choice

There are three related answers. Numerical convenience is only the third.

4.1 Independent probabilities multiply

Suppose two events A and B are independent and the model assigns them probabilities p and q . Then

$$\mathbb{P}(A \cap B) = pq.$$

If a numerical measure $I(p)$ is to represent the information or surprise of observing an event of probability p , a natural consistency requirement is

$$I(pq) = I(p) + I(q). \quad (8)$$

In words: independent evidence should add even though independent probabilities multiply.

The logarithm has exactly this property:

$$\log(pq) = \log p + \log q.$$

Because smaller probabilities should produce larger positive surprise, the sign is reversed:

$$\boxed{I(p) = -k \log p}, \quad k > 0. \quad (9)$$

Proposition 4.1 (Uniqueness of logarithmic surprise). Suppose $I : (0, 1] \rightarrow \mathbb{R}$ is continuous, $I(1) = 0$, less probable events have no less surprise, and independent surprises add as in equation (8). Then $I(p) = -k \ln p$ for some $k > 0$.

Proof sketch. Write $p = e^{-t}$ for $t \geq 0$ and define $g(t) = I(e^{-t})$. Then

$$g(s+t) = I(e^{-s}e^{-t}) = g(s) + g(t).$$

The continuous solutions to this additive functional equation are $g(t) = kt$. Consequently,

$$I(p) = g(-\ln p) = -k \ln p.$$

Monotonicity requires $k > 0$. □

Thus the logarithm is not merely a convenient curve chosen to punish mistakes. Under these ordinary requirements, its functional form is essentially forced. This is the foundation of Shannon information theory [1].

4.2 The base of the logarithm

The logarithmic *form* is fundamental; the base is a choice of units:

- $\log_2$ measures information in *bits*;
- $\ln$ measures information in *nats*;
- any other base differs only by a positive constant factor.

Since

$$\log_b p = \frac{\ln p}{\ln b},$$

changing the base rescales every loss by the same positive constant and does not change the minimizing parameters. The natural logarithm is standard in statistics and optimization because its derivatives are simplest. Therefore, “the logarithm is natural” and “the natural logarithm is mandatory” are different claims: the first is conceptually justified; the second is false.

4.3 Logarithms preserve the optimum and improve computation

The logarithm is strictly increasing, so

$$a > b \iff \log a > \log b.$$

Maximizing a positive likelihood and maximizing its logarithm therefore give exactly the same optimizer. The logarithm also turns products into sums, simplifies derivatives, and prevents products of many small probabilities from numerically underflowing to zero.

5 Maximum likelihood

Definition 5.1 (Maximum likelihood principle). Choose the parameter value that makes the observed data have the greatest probability under the assumed model:

$$\hat{\theta}_{\text{MLE}} = \arg \max_{\theta} L(\theta; \mathcal{D}) = \arg \max_{\theta} \mathbb{P}(\mathcal{D} \mid \theta).$$

5.1 Coin-flip example

Suppose a coin is flipped $n = 10$ times and the observed sequence contains $k = 8$ heads and 2 tails. If flips are independent with head probability p , the likelihood of the particular observed sequence is

$$L(p) = p^8(1-p)^2.$$

If we record only the count rather than the sequence, the binomial probability includes $\binom{10}{8}$, but that factor does not depend on p and therefore does not change the maximum-likelihood estimate.

The log-likelihood is

$$\ell(p) = 8 \log p + 2 \log(1-p).$$

Differentiating and setting the result to zero,

$$\ell'(p) = \frac{8}{p} - \frac{2}{1-p} = 0 \implies 8(1-p) = 2p \implies \hat{p} = 0.8.$$

More generally, the Bernoulli MLE is the observed fraction of ones, $\hat{p} = k/n$.

5.2 Likelihood for a logistic-regression dataset

Logistic regression normally assumes that labels are *conditionally independent given their predictors and the parameters*. Therefore,

$$L(w, b) = \prod_{i=1}^n \mathbb{P}(Y_i = y_i \mid x_i; w, b) = \prod_{i=1}^n p_i^{y_i} (1-p_i)^{1-y_i}. \quad (10)$$

Taking logarithms yields

$$\log L(w, b) = \sum_{i=1}^n [y_i \log p_i + (1-y_i) \log(1-p_i)]. \quad (11)$$

Machine learning usually expresses fitting as loss minimization rather than likelihood maximization. Negating equation (11) gives the negative log-likelihood (NLL):

$$\text{NLL}(w, b) = - \sum_{i=1}^n [y_i \log p_i + (1-y_i) \log(1-p_i)]. \quad (12)$$

Dividing by n gives the mean binary cross-entropy:

$$\boxed{\text{BCE}(w, b) = -\frac{1}{n} \sum_{i=1}^n [y_i \log p_i + (1-y_i) \log(1-p_i)]}. \quad (13)$$

For a fixed dataset and no additional penalty, the sum and mean have the same minimizer. They differ only by a factor of n . If a regularization term is added, however, the convention matters because it changes the relative scale of the data loss and penalty.

The complete logical chain

1. **Model each binary label** $Y_i | x_i \sim \text{Bernoulli}(p_i)$, with $p_i = \sigma(w^\top x_i + b)$.

2. **Combine independent observations** $L(w, b) = \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1-y_i}$.

3. **Take the logarithm** $\log L = \sum_{i=1}^n [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$.

4. **Negate and average** $\text{BCE}(w, b) = -\frac{1}{n} \log L(w, b)$.

In words: Bernoulli specifies each probability; independence creates the product; the logarithm converts that product to a sum; negation and averaging turn it into the loss minimized during training.

6 The shape of binary cross-entropy

For a positive label $y = 1$,

$$\ell(1, p) = -\ln p.$$

For a negative label $y = 0$,

$$\ell(0, p) = -\ln(1 - p).$$

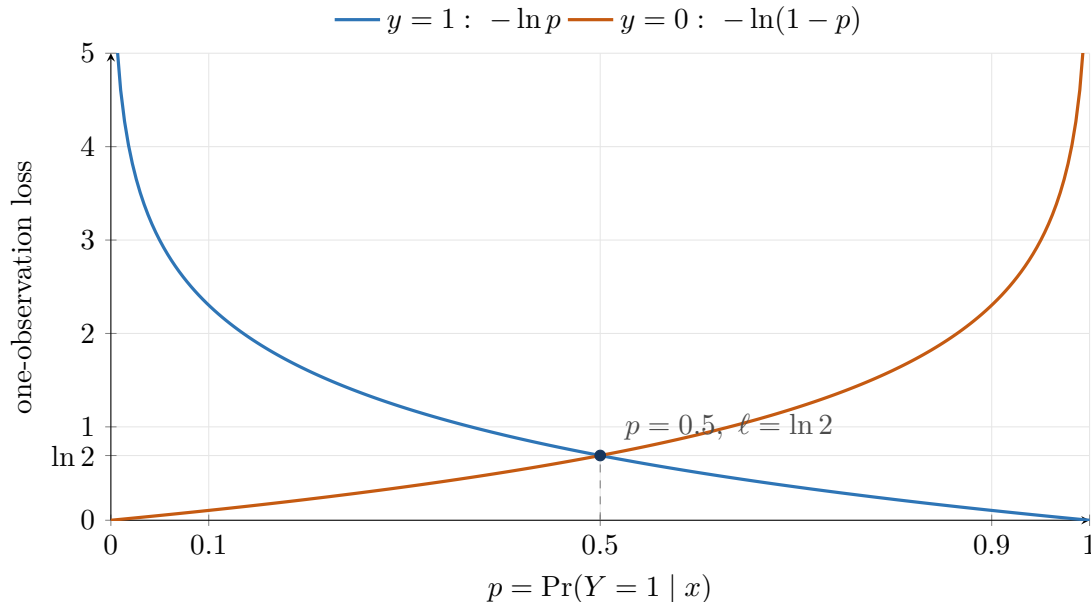


Figure 1: Binary cross-entropy as a function of the predicted class-1 probability. The two curves meet at $p = 0.5$, but this is not a minimum. Each curve reaches loss 0 only when all probability is placed on its observed class.

Important features of figure 1 are:

- If $y = 1$, the loss decreases toward 0 as $p \rightarrow 1$ and diverges as $p \rightarrow 0$.

- If $y = 0$, the loss decreases toward 0 as $p \rightarrow 0$ and diverges as $p \rightarrow 1$.
- At $p = 0.5$, both losses equal $-\ln(0.5) = \ln 2 \approx 0.693$. This is the crossing point and the point of maximum binary uncertainty, not the minimum of either curve.
- The theoretical range is $[0, \infty]$, with infinite loss when the model assigns probability zero to the outcome that occurs.

The divergence at a confidently wrong prediction is deliberate. A probabilistic model is making a much stronger and more falsifiable claim when it assigns probability 10^{-9} than when it assigns probability 0.4 to an event that then occurs.

7 A complete numerical example

Suppose three independent observations have labels and predictions as follows:

i	y_i	$p_i = \Pr(Y_i = 1 \mid x_i)$	Probability of observed label r_i	$-\ln r_i$
1	1	0.8	0.8	0.2231
2	0	0.3	$1 - 0.3 = 0.7$	0.3567
3	1	0.6	0.6	0.5108

The joint likelihood is

$$L = 0.8 \times 0.7 \times 0.6 = 0.336.$$

The negative log-likelihood is

$$-\ln L = -\ln(0.336) = 1.0906.$$

The individual log losses add to the same value:

$$0.2231 + 0.3567 + 0.5108 = 1.0906.$$

The mean BCE is

$$\frac{1.0906}{3} = 0.3635.$$

This example displays the entire product-to-sum relationship numerically.

8 From probability loss to logit loss

Substitute $p = \sigma(z)$ into the one-observation loss:

$$\begin{aligned}
 \ell(z, y) &= -y \log \sigma(z) - (1 - y) \log(1 - \sigma(z)) \\
 &= \log(1 + e^z) - yz \\
 &= \text{softplus}(z) - yz.
 \end{aligned} \tag{14}$$

Here $\text{softplus}(z) = \log(1 + e^z)$. This form reveals several useful mathematical properties.

8.1 The gradient simplifies to $p - y$

Because

$$\frac{d}{dz} \log(1 + e^z) = \frac{e^z}{1 + e^z} = \sigma(z) = p,$$

we obtain

$$\boxed{\frac{\partial \ell}{\partial z} = p - y.} \quad (15)$$

Using $z = w^\top x + b$ and the chain rule,

$$\nabla_w \ell = (p - y)x, \quad \frac{\partial \ell}{\partial b} = p - y. \quad (16)$$

For example, if $y = 1$ and $p = 0.7$, then $p - y = -0.3$. Gradient descent subtracts a negative quantity, increasing z and therefore increasing p . If $y = 0$ and $p = 0.7$, then $p - y = 0.7$; gradient descent decreases z and therefore decreases p .

8.2 Convexity

The second derivative with respect to the logit is

$$\frac{\partial^2 \ell}{\partial z^2} = p(1 - p) \geq 0. \quad (17)$$

For a dataset, the Hessian with respect to w has the form

$$\nabla_w^2 \text{NLL} = X^\top W X,$$

where W is diagonal with entries $p_i(1 - p_i) \geq 0$. Hence the unregularized logistic-regression NLL is convex in its parameters. It may fail to be *strictly* convex if the design matrix lacks rank or if other degeneracies are present.

Convexity means there are no inferior local minima in ordinary logistic regression. This favorable property belongs to the combination of a linear logit, sigmoid link, and Bernoulli negative log-likelihood.

9 Cross-entropy, entropy, and KL divergence

Optional enrichment: information theory. This section gives the information-theoretic interpretation of binary cross-entropy. The later sections do not depend on it.

The one-observation loss is a realized score. Cross-entropy in its population form averages that score over outcomes generated by a true distribution.

Let the true conditional probability of class 1 be π , while the model reports p . Then

$$Q = (1 - \pi, \pi), \quad P = (1 - p, p).$$

The binary cross-entropy is

$$H(Q, P) = \mathbb{E}_{Y \sim Q}[-\log P(Y)] = -\pi \log p - (1 - \pi) \log(1 - p). \quad (18)$$

This is the model's *expected* surprisal when reality follows Q but probabilities are quoted according to P .

The entropy of the true distribution is

$$H(Q) = -\pi \log \pi - (1 - \pi) \log(1 - \pi), \quad (19)$$

and the Kullback–Leibler divergence from Q to P is

$$D_{\text{KL}}(Q\|P) = \pi \log \frac{\pi}{p} + (1 - \pi) \log \frac{1 - \pi}{1 - p}. \quad (20)$$

These quantities satisfy

$$H(Q, P) = H(Q) + D_{\text{KL}}(Q\|P). \quad (21)$$

The entropy $H(Q)$ is the irreducible uncertainty inherent in the true outcome distribution. The nonnegative KL term is the extra expected log loss caused by using the wrong predictive distribution. Therefore, expected cross-entropy is minimized when $P = Q$, or $p = \pi$.

9.1 Why log loss rewards honest probabilities

For fixed true probability π , differentiate the expected loss in equation (18) with respect to the reported probability p :

$$\frac{\partial H(Q, P)}{\partial p} = -\frac{\pi}{p} + \frac{1 - \pi}{1 - p} = \frac{p - \pi}{p(1 - p)}.$$

The derivative is zero precisely at $p = \pi$, and this point is the unique minimum for $0 < \pi < 1$. The logarithmic score is therefore a *strictly proper scoring rule*: in expectation, a forecaster does best by reporting its true probability rather than exaggerating or concealing uncertainty.

Single observation versus population

If one observed label is $y = 1$, the empirical point-mass distribution is $Q_y = (0, 1)$, whose entropy is zero. Its cross-entropy with $P = (1 - p, p)$ is simply $-\log p$. If repeated outcomes at the same x truly have probability $\pi = 0.7$, the population cross-entropy averages both possible labels and equals $-0.7 \log p - 0.3 \log(1 - p)$.

In supervised learning, the sample mean in equation (13) is an empirical estimate of expected cross-entropy under the data-generating distribution. This is why the same expression appears as both an average training loss and an information-theoretic quantity.

10 Numerical stability

The mathematical loss is intentionally unbounded, but a naive computer implementation can create avoidable infinities or undefined values.

10.1 What goes wrong

A naive implementation performs the following steps:

$$z \longrightarrow p = \sigma(z) \longrightarrow -y \log p - (1 - y) \log(1 - p).$$

For a very negative z , finite-precision arithmetic may round p to exactly 0. If $y = 1$, the program evaluates $-\log 0 = +\infty$. For a very positive z , p may round to exactly 1, making $\log(1 - p)$ problematic when $y = 0$. In some implementations, the algebraic-looking inactive term $0 \log 0$ can also produce a ‘NaN’ even though its mathematical limiting contribution should be zero.

10.2 Stable BCE from logits

The preferred approach is to combine the sigmoid and BCE algebraically and compute directly from z :

$$\ell(z, y) = \max(z, 0) - zy + \log(1 + e^{-|z|}). \quad (22)$$

This expression is mathematically equivalent to equation (14), but it avoids exponentiating a large positive number and avoids forming probabilities that round to exactly 0 or 1. Major machine-learning libraries therefore provide operations commonly described as *binary cross-entropy with logits* or *sigmoid cross-entropy with logits*.

Clipping probabilities to $[\varepsilon, 1 - \varepsilon]$ is a simple fallback, but it changes the objective near the boundaries and can distort gradients. If logits are available, a stable fused logit formulation is preferable.

Two different numerical issues

Taking the log of a full dataset likelihood prevents a product of many ordinary probabilities from underflowing. Computing BCE directly from logits prevents $\sigma(z)$ from rounding to 0 or 1 before a logarithm is taken. These are related but distinct uses of numerical stability.

11 Why not ordinary squared error?

Squared error is not nonsensical for probability forecasts. The Brier score $(p - y)^2$ is also a strictly proper scoring rule. Nevertheless, BCE is the canonical loss for Bernoulli logistic regression because it is exactly the model's negative log-likelihood and has especially convenient optimization geometry.

	Binary cross-entropy	Squared probability error
One-observation loss	$-y \log p - (1 - y) \log(1 - p)$	$(p - y)^2$
Statistical role	Bernoulli negative log-likelihood	Brier score; a different proper scoring rule
Gradient with respect to z	$p - y$	$2(p - y)p(1 - p)$
Confident wrong prediction	Unbounded penalty	Penalty bounded by 1
Convex in linear-logit parameters	Yes	Not generally
Behavior at saturated sigmoid	Retains a large logit gradient when confidently wrong	Extra factor $p(1 - p)$ can make the gradient very small

Thus BCE is not the only conceivable probability loss. Its special status comes from the conjunction of Bernoulli likelihood, logarithmic information, additive independent evidence, proper probabilistic scoring, and clean optimization.

12 Assumptions, limitations, and extensions

12.1 Conditional independence does not mean independent features

The likelihood product in [equation \(10\)](#) assumes that the labels are independent *conditional on* the observed predictors and model parameters. It does not require the components of x to be independent. Repeated measurements, clustered observations, time series, and network data may violate the conditional-independence assumption and require a different likelihood or dependence model.

12.2 Maximum likelihood depends on the model

Maximum likelihood finds the best parameter inside the assumed model family. It does not guarantee that the family is correct, that omitted variables are harmless, or that predictions generalize. Model diagnostics, validation data, and domain knowledge remain necessary.

12.3 Complete separation

If a linear boundary perfectly separates the two classes, unregularized logistic regression may have no finite MLE: increasing the magnitude of w can push every training probability closer to its observed label and drive the BCE toward zero without attaining a finite optimum. Regularization, such as an L_2 penalty, produces a finite solution and controls model complexity. A penalized objective is no longer pure maximum likelihood; with an appropriate prior interpretation, it can be viewed as maximum a posteriori estimation.

12.4 Class weighting

Weighted BCE is often used for unequal error costs or sampling designs:

$$\ell_{\text{weighted}}(y, p) = -[\alpha y \log p + \beta(1 - y) \log(1 - p)].$$

Weights change the target criterion and can change probability calibration. They should be chosen to match the inferential or decision objective, not merely because the classes have different frequencies.

12.5 Multiclass extension

For K mutually exclusive classes, logistic regression generalizes to softmax regression. If q_k is the true one-hot label distribution and p_k is the model probability for class k , categorical cross-entropy is

$$H(Q, P) = - \sum_{k=1}^K q_k \log p_k.$$

Because Q is one-hot, this again reduces to the negative log probability assigned to the observed class.

13 Common misconceptions

1. “BCE takes the logarithm of the prediction error.”

No. It takes the negative logarithm of the probability assigned to the observed outcome.

2. “The logarithm comes from the Bernoulli distribution.”

Not directly. Bernoulli supplies $p^y(1-p)^{1-y}$. The log is applied when forming an additive log-likelihood or logarithmic score.

3. “The minimum loss is at $p = 0.5$.”

No. At $p = 0.5$, the loss is $\ln 2 \approx 0.693$ for either label. The minimum is 0 at $p = 1$ when $y = 1$ and at $p = 0$ when $y = 0$.

4. “The log is used only to avoid tiny likelihood products.”

Numerical stability matters, but the deeper reason is additivity: independent probabilities multiply while independent information should add.

5. “Natural logarithm is the only acceptable base.”

No. Any logarithm gives the same optimizer up to positive scaling. Base 2 yields bits; base e yields nats and simpler calculus.

6. “Cross-entropy is classification accuracy.”

No. Accuracy sees only which side of a decision threshold wins. Cross-entropy evaluates the full reported probability and strongly distinguishes cautious from confident errors.

7. “Likelihood is the probability that the parameter is true.”

No. It is the probability of the observed data considered as a function of the parameter. A probability distribution over parameters belongs to Bayesian inference.

8. “The loss becoming infinite means the theory is broken.”

No. Infinite loss is the coherent logarithmic penalty for assigning probability zero to an event that occurs. Stable implementations keep finite logits computable without pretending the theoretical boundary is harmless.

14 Terminology reference

Bernoulli distribution

A probability model for one binary outcome, parameterized by $p = \Pr(Y = 1)$.

Binary cross-entropy (BCE)

The average or expected negative logarithmic score for binary outcomes.

Calibration

Agreement between predicted probabilities and long-run event frequencies; among cases predicted near 0.7, roughly 70% should be positive in a well-calibrated model.

Conditional independence

Independence after conditioning on specified variables and parameters; the standard assumption that permits the logistic-regression likelihood to factor over observations.

Entropy

Expected self-information under the same distribution that generates and scores outcomes: $H(Q) = \mathbb{E}_Q[-\log Q(Y)]$.

Cross-entropy

Expected self-information when outcomes come from Q but are scored using probabilities from P : $H(Q, P) = \mathbb{E}_Q[-\log P(Y)]$.

KL divergence

The nonnegative excess cross-entropy due to using P instead of Q : $D_{\text{KL}}(Q\|P) = H(Q, P) - H(Q)$.

Likelihood

The probability of the observed data, regarded as a function of unknown model parameters.

Logit

The log-odds $\log[p/(1-p)]$; in logistic regression it equals $w^\top x + b$.

Log-likelihood

The logarithm of the likelihood; it converts products across observations into sums.

Maximum likelihood estimate (MLE)

The parameter value maximizing the likelihood, equivalently the log-likelihood.

Negative log-likelihood (NLL)

The negative log-likelihood, written as a minimization objective. For Bernoulli observations it is summed BCE.

Proper scoring rule

A probability scoring rule whose expected value is optimized by reporting the true probability; log loss and the Brier score are examples.

Sigmoid

The inverse-logit mapping $\sigma(z) = 1/(1 + e^{-z})$, which maps any real score to $(0, 1)$.

Surprisal / self-information

The realized information $-\log p$ associated with observing an event assigned probability p .

15 Summary

The core ideas can be compressed into six statements:

1. Logistic regression models $Y \mid x$ as Bernoulli with probability $p = \sigma(w^\top x + b)$.
2. The Bernoulli formula gives the observed label probability $p^y(1-p)^{1-y}$.
3. Conditional independence makes dataset likelihood contributions multiply.
4. Requiring independent information to add leads essentially uniquely to $-\log p$; the logarithm also turns the likelihood product into a sum without changing its maximizer.
5. The resulting Bernoulli negative log-likelihood is binary cross-entropy, and its logit derivative is the simple residual $p - y$.
6. Stable software computes the loss directly from logits, not by separately forming a possibly rounded probability and then taking its logarithm.

The deepest conceptual point is therefore not that a logarithm was arbitrarily attached to an error. The model assigns probabilities through a conditional Bernoulli distribution; maximum likelihood asks those probabilities to support the observed data; and logarithmic information is the additive language in which independent probabilistic evidence is naturally measured.

A Manifestations of BCE Across Machine-Learning Algorithms

Binary cross-entropy is not tied to one algorithm. It is tied to a *probability model*: somewhere in the learning problem, an outcome is represented as a Bernoulli event and the model produces a probability for that event. Logistic regression is the simplest example, but the logit can be produced by a neural network, an additive ensemble of trees, a matrix-factorization model, or another function approximator.

Terminology: BCE and BCE loss

Binary cross-entropy and *binary cross-entropy loss* normally refer to the same objective. The common forms are **BCE** and **BCE loss**; the latter is linguistically redundant but conventional in software documentation.

A.1 One mathematical template

Suppose a learning problem contains binary targets indexed by pairs (i, j) , where i identifies an observation and j might identify a label, pixel, item, time interval, or detection candidate. A model produces logits

$$z_{ij} = f_{\theta}(x_i)_j, \quad p_{ij} = \sigma(z_{ij}),$$

and applies the elementwise Bernoulli loss

$$\ell_{ij} = -y_{ij} \log p_{ij} - (1 - y_{ij}) \log(1 - p_{ij}). \quad (23)$$

Training aggregates the applicable terms:

$$\mathcal{L}(\theta) = \frac{1}{|\mathcal{I}|} \sum_{(i,j) \in \mathcal{I}} \ell_{ij}, \quad (24)$$

possibly with weights or masks. The same loss therefore takes on different practical meanings according to:

1. what binary event y_{ij} represents;
2. how the algorithm constructs the logit z_{ij} ; and
3. whether BCE is the direct training objective, a local split criterion, or only an evaluation metric.

Using a sum of elementwise BCE terms amounts to a factorized Bernoulli likelihood for those targets, at least as a working objective. That factorization may be an exact scientific assumption, a useful conditional approximation, or simply a surrogate training criterion. These interpretations should not be silently conflated.

A.2 Direct uses of BCE

Table 1: Common direct uses of binary cross-entropy.

Setting	Binary event	How BCE manifests
Classical logistic regression	Entire observation is class 1 or 0	The logit is linear, $z = w^\top x + b$. BCE is exactly the conditional Bernoulli NLL.
Binary neural-network classifier	Entire observation is positive or negative	A nonlinear network produces features and a final scalar logit. The output layer is logistic regression on learned features.
Multi-label classification	Attribute j is present or absent	The model produces one sigmoid per label and sums or averages BCE across labels. Labels need not be mutually exclusive, unlike softmax classes.
Binary semantic segmentation	Pixel j belongs or does not belong to a target region	A convolutional or transformer model produces a logit for each pixel. Pixelwise BCE is often combined with Dice- or overlap-based losses.
Object detection	An object is present at a candidate location, or an independent class is present	BCE can score objectness and independent class outputs. Class imbalance often motivates weighting or focal loss.
Click-through and conversion prediction	User clicks, purchases, or converts	A linear model, neural model, or factorization model produces an interaction logit. BCE is the event NLL when the training sample reflects the intended population.
Implicit-feedback recommendation	User interacts or does not interact with an item	Positive interactions and sampled negatives are trained with logistic terms. Negative sampling changes the sampled class distribution, so raw scores need not be calibrated population probabilities.
Bernoulli decoder or autoencoder	A reconstructed bit or binary feature equals 1 or 0	Decoder outputs parameterize Bernoulli observations. This likelihood is natural for binary data, but it may be a poor observation model for genuinely continuous measurements.
Classic GAN discriminator	Input is real or generated	The discriminator is trained as a binary classifier with logarithmic loss. Common generator objectives use related log terms but have a different optimization role [9].
Discrete-time survival model	An event occurs or does not occur during interval j , conditional on survival to that interval	Interval-specific sigmoid outputs estimate hazards. The observed or censored survival likelihood decomposes into BCE-like event and non-event terms over the intervals at risk.
Negative-sampling objectives	A data pair is observed or sampled from a noise distribution	Logistic discrimination between positive and sampled negative pairs produces a sum of BCE-like log-sigmoid terms; word-embedding objectives provide a familiar example [10].

The phrase “BCE is used” is strongest when [equation \(24\)](#) is the explicit objective. It is weaker when an algorithm merely produces probabilities that are later evaluated by BCE.

A.3 Ordinary decision trees: entropy as fitted leaf BCE

Consider a candidate tree node containing n_1 positive and n_0 negative observations, with $n = n_0 + n_1$ and positive fraction

$$q = \frac{n_1}{n}.$$

If the node becomes a leaf and predicts one constant probability p for all of its observations, its total BCE is

$$\mathcal{L}_{\text{node}}(p) = -n_1 \log p - n_0 \log(1 - p). \quad (25)$$

Differentiating shows that the best constant probability is the empirical class fraction:

$$\frac{d\mathcal{L}_{\text{node}}}{dp} = -\frac{n_1}{p} + \frac{n_0}{1-p} = 0 \implies \hat{p} = q.$$

At this fitted value, the average BCE in the node is

$$\frac{1}{n} \mathcal{L}_{\text{node}}(\hat{p}) = -q \log q - (1 - q) \log(1 - q) = H(q), \quad (26)$$

which is exactly binary entropy.

For a proposed split into left and right children, entropy-based information gain is

$$\Delta H = H(q_{\text{parent}}) - \frac{n_L}{n} H(q_L) - \frac{n_R}{n} H(q_R). \quad (27)$$

Consequently, maximizing information gain is equivalent to maximizing the reduction in training BCE obtained when the parent and candidate children each use their own fitted constant probability. This explains the precise link between the entropy split criterion and BCE in classification trees [6].

There are two important qualifications:

- This is a *local greedy split calculation*. An ordinary decision tree does not differentiate a smooth, global BCE objective through every possible tree topology.
- Many trees use Gini impurity, $1 - q^2 - (1 - q)^2 = 2q(1 - q)$, rather than entropy. Gini and entropy have similar qualitative shapes, but Gini is not BCE and contains no logarithm.

A.4 Random forests and probability evaluation

A random forest trains many trees on perturbed data and feature sets, then averages their class votes or leaf probabilities. Depending on the implementation, each tree may use entropy, Gini impurity, or another split rule. The forest therefore does not usually minimize one global BCE objective.

Nevertheless, BCE remains relevant in three ways:

1. an entropy-trained constituent tree uses the local relationship in [equation \(26\)](#);
2. the forest's final probabilities can be selected or evaluated using held-out log loss; and
3. calibration methods can be applied when averaged tree probabilities are systematically too sharp or too conservative.

A raw leaf estimate $\hat{p} = n_1/n$ can equal exactly 0 or 1. It gives zero training loss in a pure leaf but infinite test log loss if the opposite class later appears. The symmetric Beta smoothing rule $\tilde{p} = (n_1 + \alpha)/(n + 2\alpha)$ keeps estimates away from the boundaries. Common choices include $\alpha = 1$ for Laplace smoothing and $\alpha = 1/2$ for Jeffreys smoothing. Cross-validation or a dedicated calibration set is preferable when enough data are available.

A.5 Gradient-boosted trees: direct optimization of BCE

Binary gradient boosting constructs an additive *logit* model

$$F_M(x) = F_0(x) + \sum_{m=1}^M \eta f_m(x), \quad p(x) = \sigma(F_M(x)), \quad (28)$$

where each f_m is a regression tree and η is a learning rate. The ensemble can directly minimize

$$\sum_{i=1}^n [-y_i \log p_i - (1 - y_i) \log(1 - p_i)].$$

For the current logit $F(x_i)$, the first and second derivatives are

$$g_i = p_i - y_i, \quad h_i = p_i(1 - p_i). \quad (29)$$

First-order boosting fits a new tree to approximate the negative gradient $y_i - p_i$. Newton-style boosting uses both g_i and h_i to form split gains and leaf updates. This is a direct algorithmic manifestation of the same $p - y$ gradient derived for logistic regression. The difference is that the logit function is now built from trees rather than restricted to a linear form [7, 8].

Tree method	Relationship to BCE	Key distinction
Entropy classification tree	Local equivalence	Split gain equals reduction in fitted node BCE.
Gini classification tree	Indirect / none algebraically	Similar impurity goal, but no logarithmic loss.
Random forest	Usually evaluation-level	Trees are bagged; the ensemble generally has no single global BCE training objective.
Logistic gradient boosting	Direct global objective	Trees add corrections to logits using BCE gradients, and often Hessians.
AdaBoost	Different objective	Classical AdaBoost is associated with exponential loss, not BCE.

A.6 Multi-label learning versus multiclass learning

This distinction is a frequent source of implementation errors.

Multi-label problem. Several labels may simultaneously be true, such as an image containing both a dog and a car. The model uses independent sigmoid outputs and elementwise BCE:

$$p_k = \sigma(z_k), \quad \mathcal{L} = - \sum_{k=1}^K [y_k \log p_k + (1 - y_k) \log(1 - p_k)].$$

The probabilities do not need to sum to 1.

Single-label multiclass problem. Exactly one of K classes is true, such as a digit being exactly one of 0 through 9. The usual model uses a softmax distribution

$$p_k = \frac{e^{z_k}}{\sum_{r=1}^K e^{z_r}}, \quad \sum_{k=1}^K p_k = 1,$$

and categorical cross-entropy. Applying independent BCE terms instead changes the probabilistic model. A one-vs-rest construction can sometimes be useful, but its outputs are not automatically a coherent mutually exclusive probability distribution.

A.7 Weighted BCE, focal loss, masks, and soft targets

Many practical objectives are modifications of BCE rather than the plain Bernoulli NLL.

Weighted BCE. Different costs or sampling weights can be represented as

$$\ell_{\text{weighted}} = -\alpha y \log p - \beta(1 - y) \log(1 - p). \quad (30)$$

This can emphasize rare positives or asymmetric errors, but arbitrary weighting changes the population objective. If calibrated probabilities are required, class-prior changes and sampling weights must be handled deliberately.

Focal loss. Define the probability assigned to the observed class as

$$p_t = yp + (1 - y)(1 - p).$$

Binary focal loss is

$$\text{FL}(p_t) = -\alpha_t(1 - p_t)^\gamma \log p_t. \quad (31)$$

The factor $(1 - p_t)^\gamma$ downweights already-easy examples and concentrates learning on difficult ones. With $\gamma = 0$, focal loss reduces to weighted BCE. It is widely associated with dense object detection [11].

Masked BCE. If only some targets are observed or meaningful, a mask $m_{ij} \in \{0, 1\}$ limits the reduction to valid elements:

$$\mathcal{L} = \frac{\sum_{i,j} m_{ij} \ell_{ij}}{\sum_{i,j} m_{ij}}.$$

This is common for padded sequences, partially annotated labels, and spatial regions that should be ignored.

Soft targets. The algebra permits $y \in [0, 1]$ rather than only $\{0, 1\}$. Such targets can encode label smoothing, uncertain annotations, distillation probabilities, or mixtures of examples. In that case the term is cross-entropy between two Bernoulli distributions rather than the NLL contribution of one hard observed label.

A.8 When BCE is not the appropriate likelihood

BCE should be chosen because the output represents a Bernoulli event or because logarithmic binary scoring is deliberately desired, not merely because the word “classification” appears in the problem.

Outcome structure	Typical model or loss	Why plain BCE may be unsuitable
One of $K > 2$ mutually exclusive classes	Softmax categorical cross-entropy	Independent Bernoulli outputs do not enforce probabilities summing to 1.
Count data	Poisson or negative-binomial NLL	Counts are not binary outcomes.
Continuous measurements	Gaussian NLL / squared error, or another continuous likelihood	A Bernoulli observation model has the wrong support.
Margin-based classification	Hinge loss	The goal is a separating margin rather than a calibrated Bernoulli likelihood.
Classical AdaBoost	Exponential loss	Its stagewise derivation uses a different surrogate objective.
Ranking	Pairwise or listwise ranking loss	The target concerns relative ordering rather than an isolated binary probability.
Dependent structured outputs	Structured likelihood or task-specific loss	Independent elementwise BCE may ignore essential dependence among outputs.

A.9 Evaluation and interpretation across algorithms

BCE can evaluate any model that outputs binary probabilities, even if BCE was not its training objective. This includes random forests, Bayesian models, nearest-neighbor probability estimates, and calibrated margin classifiers. However, several cautions apply:

- **Accuracy and BCE answer different questions.** Accuracy evaluates thresholded decisions; BCE evaluates the entire probability assignment.
- **AUC and BCE answer different questions.** AUC primarily measures ranking; BCE is sensitive to probability calibration as well as discrimination.
- **The prevalence baseline matters.** A constant model predicting the positive fraction q has average BCE $H(q)$. Loss values from datasets with different prevalences are therefore not automatically comparable.
- **Sampling can alter probability meaning.** Case-control sampling, negative sampling, or rebalancing can yield useful classifiers while shifting the relationship between raw scores and population probabilities.

- **Boundary probabilities are hazardous.** Predictions of exactly 0 or 1 receive infinite loss when contradicted. Stable logit computation addresses arithmetic failure; smoothing and calibration address overly extreme statistical estimates.

A useful classification of BCE manifestations

Across machine learning, BCE plays four distinguishable roles:

1. **Exact likelihood:** Bernoulli logistic regression, Bernoulli neural outputs, and discrete hazards.
2. **Direct surrogate objective:** logistic gradient boosting, binary discrimination, and some negative-sampling objectives.
3. **Local impurity measure:** entropy-based classification-tree splitting.
4. **Evaluation score:** assessment of probabilistic predictions from almost any binary classifier.

Recognizing the role prevents an important category error: the same numerical formula can be an assumed data likelihood in one algorithm, an optimization surrogate in another, and merely an external score in a third.

References

- [1] C. E. Shannon, “A Mathematical Theory of Communication,” *Bell System Technical Journal*, vol. 27, pp. 379–423 and 623–656, 1948.
- [2] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.
- [3] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. Springer, 2009.
- [4] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [5] K. P. Murphy, *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [6] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*. Wadsworth, 1984.
- [7] J. H. Friedman, “Greedy Function Approximation: A Gradient Boosting Machine,” *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [8] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.
- [9] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, “Generative Adversarial Nets,” in *Advances in Neural Information Processing Systems 27*, 2014.
- [10] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, “Distributed Representations of Words and Phrases and their Compositionality,” in *Advances in Neural Information Processing Systems 26*, 2013.
- [11] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal Loss for Dense Object Detection,” in *Proceedings of the IEEE International Conference on Computer Vision*, pp. 2980–2988, 2017.